Network Fix & Redeployment Report

Issues, Fixes & Verification Log

FinClaw — AI-Assisted Paper Trading Dashboard

1. Project Overview

This report documents all known issues found in the original FinClaw codebase, the fixes applied, verification steps taken, and remaining limitations.

2. Issues Found & Fixes Applied

#	Issue	Severity	Fix Applied
1	Plaintext passwords stored in database	Critical	Added bcrypt hashing in security.py; legacy plain passwords auto-migrated on r
2	Real-looking secrets in .env.example	High	Replaced all values with empty placeholders
3	No input validation on any endpoint	High	Pydantic v2 schemas with field constraints on all request bodies
4	Unhandled exceptions returned raw 500 errors	Medium	Global exception_handler returns structured JSON {"ok":false,"detail":"..."}
5	No audit trail for trade decisions	Medium	AuditLog table records every trade intent with decision and reason
6	Alpaca SDK import crash if not installed	Medium	Try/except import block; graceful simulated fallback mode
7	CORS set to wildcard *	Low	Configurable via CORS_ORIGINS env var; wildcard only in dev
8	No .gitignore for sensitive files	Low	Added rules for .env, *.db, __pycache__, .venv
9	Portfolio data fully mocked	Info	Documented as demo limitation; real broker integration is optional
10	Chat AI uses keyword matching only	Info	Documented; LLM integration left as future improvement

3. Security Improvements Detail

3.1 Password Hashing

```
# security.py
from passlib.context import CryptContext
pwd_context = CryptContext(schemes=["bcrypt"])
def hash_password(plain: str) -> str:
    return pwd_context.hash(plain)
def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)
```

3.2 Input Validation Example

```
# schemas.py
class TradeIntent(BaseModel):
    ticker: str = Field(min_length=1, max_length=20)
    side: str  # validated to "buy" or "sell" only
    notional_usd: float = Field(gt=0, le=1_000_000)
    @field_validator("side")
    def validate_side(cls, v):
        if v.strip().lower() not in {"buy","sell"}:
            raise ValueError("side must be buy or sell")
        return v.strip().lower()
```

3.3 Global Error Handler

```
# main.py
@app.exception_handler(Exception)
async def unhandled_exception_handler(request, exc):
    logger.exception("Unhandled error for %s %s", request.method, request.url.path)
    return JSONResponse(status_code=500,
        content={"ok": False, "detail": "Internal server error"})
```

4. Verification Steps

Test	Command / Action	Expected Result
Health check	GET /health	{"ok":true,"app":"FinClaw API"}
Onboarding	POST /api/auth/onboarding with valid payload	{"ok":true,"user":{...}}
Login valid	POST /api/auth/login correct credentials	{"ok":true,"user":{...}}
Login invalid	POST /api/auth/login wrong password	HTTP 401
Trade allowed	POST /api/trade-intents AAPL $50	{"ok":true,"trade_id":...}
Trade blocked — bad ticker	POST /api/trade-intents XYZ $50	HTTP 403 ticker not approved
Trade blocked — over limit	POST /api/trade-intents AAPL $200	HTTP 403 exceeds max notional
Daily limit	4th trade same day	HTTP 403 daily limit reached
Pytest suite	cd backend && pytest	All tests pass
Agent run-once	POST /api/agent/run-once	Cycle executes, trade logged

5. Redeployment Steps After Fix

```
# 1. Pull latest code
git pull
# 2. Reinstall dependencies
pip install -r requirements.txt
# 3. Reset database (if schema changed)
del finclaw.db
python seed.py
# 4. Restart server
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
# 5. Run tests
pytest
```

6. Remaining Limitations

Area	Limitation	Recommended Fix
Auth	No JWT or session tokens	Implement OAuth2 + JWT
Auth	No CSRF protection	Add CSRF middleware for production
Database	SQLite not concurrent-safe	Migrate to PostgreSQL
Portfolio	Mocked balance and holdings	Connect to real broker account API
Chat AI	Keyword matching only	Integrate LLM (e.g. Amazon Bedrock)
Frontend	CDN-based, no bundler	Use Vite or Next.js for production
Observability	Basic logging only	Add CloudWatch / Datadog / Sentry

7. Conclusion

The fixes applied bring FinClaw to a safe, runnable demo/staging state. Password security, input validation, error handling, and audit logging are all in place. The system is deployment-ready for demo use

but requires another round of work — covering JWT auth, PostgreSQL, real broker integration, and observability — before handling real-money users in production.